W2

ENCAPSULATION

Data Structures & Object-Oriented Programming



Table of Contents

- Access Modifiers
- @property
- @property.setter
- Common Mistakes

ปัญหาที่ยังค้างอยู่จากสัปดาห์ที่ 1

สัปดาห์ที่แล้ว

- OOP ช่วยให้ข้อมูลและพฤติกรรมอยู่ด้วยกันในที่เดียว
- เราสร้าง BankAccount ที่มี method deposit() และ withdraw() พร้อม validation แล้ว
- แต่ยังมีปัญหาที่แก้ไม่ได้ :

```
account = BankAccount("Alice", 1000.0)

# นิสิตคนหนึ่งต้องการ "แก้ไขข้อมูล" โดยตรง
account.balance = -99999.0
account.owner   = "Hacker"

print(account.get_balance())
```

Python ไม่ขัดขวาง ไม่มี error เลย

เปลี่ยนเจ้าของบัญชีได้เองตามใจชอบ

-99999.0 — ข้อมูลเสียหายโดยสมบูรณ์

- เราออกแบบ deposit() และ withdraw() ให้มี validation แต่ ไม่มีอะไรบังคับให้ต้องใช้ method เหล่านั้น
  - ใครก็สามารถเข้าถึงและแก้ balance โดยตรงได้ตลอดเวลา

Encapsulation แก้ปัญหานี้โดยการ ซ่อน ข้อมูลภายใน object ไม่ให้เข้าถึงได้โดยตรง และกำหนด ช่องทางที่ควบคุมได้ สำหรับการอ่านและเขียนข้อมูลนั้น

Access Modifiers

Access Modifiers ใน Python

- ภาษาส่วนใหญ่เช่น Java หรือ C++ มี keyword เช่น private และ public บังคับ access อย่างเข้มงวด
- Python เลือกแนวทางที่ต่างออกไปโดยใช้ ธรรมเนียมการตั้งชื่อ (naming convention) เป็นสัญญาณ และ name mangling เป็นกลไกบางส่วน
- Python มี access modifier 3 ระดับ:
  - **Public** — attribute หรือ method ปกติที่ตั้งชื่อโดยไม่มี underscore นำหน้า
    - เข้าถึงได้จากทุกที่โดยไม่มีข้อจำกัด ตัวอย่างเช่น self.name
  - **Protected** — ตั้งชื่อด้วย underscore นำหน้าหนึ่งตัว เช่น self._owner
    - สัญญาณนี้บอกนักพัฒนาคนอื่นว่า "attribute นี้ใช้ภายใน class นี้หรือ subclass เท่านั้น อย่าเข้าถึงจากภายนอก"
    - แต่ Python ไม่ได้บล็อค การเข้าถึงจริงๆ — มันเป็นแค่ convention ที่ทุกคนควรเคารพ
  - **Private** — ตั้งชื่อด้วย underscore สองตัวนำหน้า เช่น self.__balance
    - Python เปลี่ยนชื่อ attribute นี้เบื้องหลังด้วยกลไกที่เรียกว่า name mangling ซึ่งทำให้เข้าถึงจากภายนอก class ด้วยชื่อเดิมไม่ได้

Name Mangling

- เมื่อ Python เห็น self._balance ภายใน class BankAccount มันจะเปลี่ยนชื่อ attribute นั้นในหน่วยความจำเป็น _BankAccount__balance โดยอัตโนมัติ
  - กระบวนการนี้เรียกว่า name mangling
- ผลที่ได้คือเมื่อเข้าถึงจากภายนอก class ด้วยชื่อเดิม Python จะหา attribute ชื่อ __balance ไม่เจอและแจ้ง AttributeError:

```
account = BankAccount("Alice", 1000.0)

print(account._owner)
print(account.__balance)
```

ได้ค่า "Alice" — protected บล็อคไม่ได้จริง

AttributeError: 'BankAccount' object has no attribute '__balance'

ในทางเทคนิค ยังสามารถเข้าถึงได้ด้วยชื่อที่ถูก mangle ว่า account._BankAccount__balance
แต่ถือเป็นการ ละเมิดกฎ อย่างชัดเจน และไม่ควรทำในโค้ดจริง

สรุป Name Mangling

- กลไกที่ Python ใช้เพื่อป้องกันไม่ให้โค้ดภายนอก class เข้าถึง attribute ที่ผู้ออกแบบตั้งใจให้เป็น "private" ได้โดยตรง
- เมื่อเราตั้งชื่อ attribute ขึ้นต้นด้วย __ (double underscore) สองตัว เช่น self.__balance
  - Python จะ เปลี่ยนชื่อ attribute นั้นโดยอัตโนมัติ ในหน่วยความจำ
  - จากเดิม __balance เป็น _ClassName__balance โดยที่ผู้เขียนโค้ดไม่ต้องทำอะไรเพิ่มเติม
  - กระบวนการเปลี่ยนชื่อนี้คือสิ่งที่เรียกว่า **Name Mangling**
- เมื่อ Python เปลี่ยนชื่อ __balance เป็น _BankAccount__balance โดยอัตโนมัติ
  - ผลที่ตามมาคือชื่อ __balance แบบเดิมไม่มีอยู่จริงในหน่วยความจำอีกต่อไป
  - ดังนั้นโค้ดภายนอกที่พยายามเข้าถึงด้วยชื่อ __balance จะได้รับ AttributeError ทันที เพราะ Python หาชื่อนั้นไม่พบ
- กลไกนี้ทำงานเฉพาะกับโค้ดที่อยู่ นอก class เท่านั้น โค้ด ภายใน class เดียวกัน ยังคงใช้ self.__balance ได้ตามปกติ
  - เราจึงไม่จำเป็นต้องพิมพ์ชื่อยาวๆ อย่าง self._BankAccount__balance ให้ยุ่งยากเมื่อเขียนอยู่ภายใน class

@property

@property — Method ที่ดูเหมือน Attribute

- เมื่อเราทำให้ attribute เป็น private ด้วย _balance เพื่อป้องกันไม่ให้โค้ดภายนอกแก้ไขโดยตรง
  - ทำให้เกิดปัญหาใหม่ คือ โค้ดภายนอกไม่สามารถ อ่านค่า ได้
- วิธีแก้เบื้องต้นที่เรานึกถึงคือการเขียน method ชื่อ get_balance() ซึ่งใช้งานได้จริง แต่มีข้อเสียด้านความสวยงามและความเป็นธรรมชาติของโค้ด ดังนี้

วิธีเดิม — ใช้ได้ แต่ดูเทอะทะ

```
print(alice.get_balance())
if alice.get_balance() > 1000:
    ...
```

วิธีที่ต้องการ — อ่านได้เป็นธรรมชาติกว่ามาก

```
print(alice.balance)
if alice.balance > 1000:
    ...
```

@property คือ กลไกที่ทำให้ได้ทั้งสองอย่างพร้อมกัน

ยังคงป้องกันข้อมูลด้วย private attribute แต่เปิดให้อ่านค่าได้ด้วย syntax ที่ดูเหมือน attribute ธรรมดา โดยไม่ต้องใช้วงเล็บ

@property

- คือ decorator ที่บอกให้ Python รู้ว่า "method นี้ให้เรียกใช้โดยไม่ต้องใส่วงเล็บได้"
- เมื่อเขียน **alice.balance** Python จะแอบเรียก method balance(self) ให้โดยอัตโนมัติ
- คนที่อ่านโค้ดจากภายนอกจึงเห็นเหมือนกำลังเข้าถึง attribute แต่จริงๆ แล้ว method กำลังทำงานอยู่เบื้องหลัง

```python
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self._owner    = owner
        self.__balance = balance  เก็บจริงใน private attribute

    @property
    def balance(self) -> float:   method แต่เรียกใช้เหมือน attribute
        """อ่านยอดเงิน"""
        return self.__balance
```

เมื่อมี @property แล้ว เราเรียกใช้ได้ดังนี้:

```python
account = BankAccount("Alice", 1000.0)
print(account.balance)  เรียกใช้โดยไม่มีวงเล็บ → 1000
print(account.balance())  TypeError: 'float' object is not callable
```

ใช้ @property เมื่อไหร่

- @property เหมาะสมเมื่อต้องการเปิดให้โค้ดภายนอกอ่านค่า attribute ที่เป็น private ได้ โดยยังคงควบคุมการเข้าถึงอยู่
- หรือเมื่อต้องการคำนวณค่าบางอย่างจาก attribute ที่มีอยู่แล้วแบบ on-the-fly โดยไม่ต้องเก็บค่าใหม่แยกต่างหาก
- ตัวอย่างที่ดีคือการแปลงหน่วย

```python
class Temperature:
    def __init__(self, celsius: float):
        self.__celsius = celsius

    @property
    def celsius(self) -> float:
        return self.__celsius

    @property
    def fahrenheit(self) -> float:              # คำนวณจาก celsius ทุกครั้งที่เรียก
        return self.__celsius * 9/5 + 32        # ไม่ต้องเก็บ fahrenheit แยกไว้

t = Temperature(100)
print(t.fahrenheit)   # 212.0 — ได้ค่าทันทีโดยไม่มีวงเล็บ
```

- @property ไม่เหมาะกับ method ที่มีการคำนวณหนัก ใช้เวลานาน หรือมี side effects เช่น การเขียนไฟล์หรือเรียก API
  - เนื่องจากผู้อ่านโค้ดมักคาดหวังว่าการเข้าถึง attribute จะเป็นการดำเนินการที่เร็วและไม่มีผลข้างเคียง
- หากมีการคำนวณหนักควรใช้ method ธรรมดาพร้อมวงเล็บแทน เพื่อสื่อสารให้ชัดเจนว่ามีการประมวลผลเกิดขึ้น

@property.setter

- @property อย่างเดียวทำให้อ่านได้อย่างเดียว การเพิ่ม setter ทำให้ เขียนค่าผ่านจุดเดียว ที่มี validation รองรับ

```python
@balance.setter
    def balance(self, value: float) -> None:
        """ตั้งค่ายอดเงิน – ต้องไม่ติดลบ"""
        if value < 0:
            raise ValueError(f"ยอดเงินต้องไม่ติดลบ ได้รับ: {value}")
        self.__balance = value
```

- สังเกตว่า setter ใช้ raise ValueError แทน print เพราะ setter เป็น low-level mechanism ที่ต้องบอกผู้เรียก (caller) อย่างชัดเจนว่า "ค่านี้ไม่ยอมรับ" ต่างจาก print ที่แค่แสดงข้อความแล้วทำงานต่อ
- เมื่อ raise แล้ว โปรแกรมจะหยุดทำงาน ณ จุดนั้นทันทีหากไม่มีการจัดการ error (ด้วย try-except)

@property.setter (2)

- เมื่อมี setter แล้ว account.balance = 500 จะเรียก setter โดยอัตโนมัติ:

```
account.balance = 500.0    # เรียก setter → self.__balance = 500.0
account.balance = -100.0   # เรียก setter → ValueError ทันที
```

- จุดสำคัญ: เมื่อใช้ setter ใน method อื่นด้วย เช่นใน deposit()
- เขียน self.balance += amount แทน self.__balance += amount
- ทำให้ validation ทำงานทุกครั้งที่ค่าเปลี่ยน ไม่ว่าจะเปลี่ยนจากที่ใด

- setter ไม่จำเป็นสำหรับ attribute ที่ควรกำหนดได้เพียงครั้งเดียวตอนสร้าง object แล้วไม่ควรเปลี่ยนแปลงอีก
  - เช่น หมายเลขบัญชีหรือวันที่เปิดบัญชี
- กรณีเหล่านี้ควรใช้ read-only property ที่จะอธิบายในหัวข้อถัดไปแทน

@property.setter (3)

- หลังจากกำหนด @property สำหรับการอ่านแล้ว หากต้องการให้โค้ดภายนอกสามารถ แก้ไขค่าได้ แต่ต้องผ่านการตรวจสอบก่อนเสมอ ก็ต้องกำหนด setter คู่กัน
- setter คือ method ที่ถูกเรียกอัตโนมัติเมื่อมีการกำหนดค่าด้วยเครื่องหมาย =
  - เช่น alice.balance = 500
  - แทนที่ค่าจะถูกเขียนลงไปตรงๆ Python จะส่งค่า 500 เข้ามาใน setter ก่อน ทำให้ผู้พัฒนาสามารถตรวจสอบความถูกต้องได้ก่อนที่จะบันทึกจริง
  - นี่คือเหตุผลที่เรียก setter ว่า "จุดศูนย์กลางของ Validation" เพราะ ทุกการแก้ไขค่าต้องผ่านจุดนี้เสมอ ไม่มีทางเลี่ยง

```python
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self._owner = owner
        self.__balance = 0.0
        self.balance = balance        # เรียกผ่าน setter ตั้งแต่ __init__

    @property
    def balance(self) -> float:
        return self.__balance

    @balance.setter
    def balance(self, value: float) -> None:
        if not isinstance(value, (int, float)):
            raise TypeError("ยอดเงินต้องเป็นตัวเลข")
        if value < 0:
            raise ValueError("ยอดเงินต้องไม่ติดลบ")
        self.__balance = value        # บันทึกจริงเมื่อผ่านการตรวจสอบแล้วเท่านั้น

alice = BankAccount("Alice", 1000)
alice.balance = 500                   # ผ่าน setter → ตรวจสอบแล้วบันทึก
alice.balance = -100                  # ValueError: ยอดเงินต้องไม่ติดลบ
```

Read-only Property

- หาก property มีแค่ @property โดยไม่มี setter มันจะเป็น read-only — อ่านได้แต่เขียนตรงๆ ไม่ได้:

```
@property
    def owner(self) -> str:
        """อ่านชื่อเจ้าของได้ แต่เปลี่ยนโดยตรงไม่ได้"""
        return self._owner
```

```
print(account.owner)        # "Alice"  ← อ่านได้
account.owner = "Bob"       # AttributeError: can't set attribute
```

Read-only property เหมาะสำหรับข้อมูลที่ไม่ควรเปลี่ยนได้จากภายนอก เช่น ชื่อเจ้าของบัญชี หรือวันที่สร้างบัญชี แต่ยังต้องอ่านได้อยู่

Read-only Property

- Read-only property คือ @property ที่กำหนดโดยไม่มี setter คู่กัน
- ผลลัพธ์คือโค้ดภายนอกสามารถ อ่านค่าได้ แต่ ไม่สามารถแก้ไขได้เลย
- Python จะ raise AttributeError ทันทีหากมีการพยายามกำหนดค่า

```python
class BankAccount:
    def __init__(self, owner: str, account_number: str):
        self.__owner = owner
        self.__account_number = account_number    # ไม่ควรเปลี่ยนหลังสร้างแล้ว

    @property
    def account_number(self) -> str:
        return self.__account_number
        # ไม่มี setter — จบแค่นี้

alice = BankAccount("Alice", "ACC-0001")
print(alice.account_number)           # "ACC-0001" — อ่านได้ปกติ
alice.account_number = "ACC-9999"     # AttributeError — แก้ไขไม่ได้
```

read-only property สื่อว่า "อ่านได้อย่างเดียว ไม่มีเหตุผลใดที่ควรแก้ไขจากภายนอก" การเลือกใช้ให้ถูกประเภทจึงเป็นการสื่อสารเจตนาของการออกแบบให้ผู้อ่านโค้ดเข้าใจได้ทันที

ตัวอย่างที่ควรเป็น Read-only

- ข้อมูลที่เหมาะกับ read-only property คือ ข้อมูลที่ถูกกำหนดตอนสร้าง object และไม่ควรเปลี่ยนแปลงตลอดอายุการใช้งาน
  - เช่น หมายเลขบัญชี วันเกิด หรือรหัสพนักงาน
- นอกจากนี้ยังใช้ได้ดีกับค่าที่คำนวณได้จาก attribute อื่นและไม่ต้องการให้ตั้งค่าโดยตรง

```python
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self.__owner = owner
        self.__balance = balance
        self.__transaction_count = 0

    @property
    def owner(self) -> str:                # ชื่อเจ้าของไม่ควรเปลี่ยน
        return self.__owner

    @property
    def transaction_count(self) -> int:    # นับจากระบบ ไม่ควรให้ตั้งเองได้
        return self.__transaction_count

    def deposit(self, amount: float) -> None:
        self.__balance += amount
        self.__transaction_count += 1      # เปลี่ยนได้เฉพาะจากภายใน class
```

ทำไม Setter ใน __init__ จึงสำคัญ

- สังเกตว่า __init__ เขียน self.balance = balance ไม่ใช่ self.__balance = balance ความแตกต่างนี้มีผลจริงในทางปฏิบัติ:

```python
# ถ้า __init__ เลี่ยง setter
class BankAccountBad:
    def __init__(self, owner: str, balance: float = 0.0):
        self._owner    = owner
        self.__balance = balance  # ← bypass setter โดยตรง

# ถ้า __init__ ใช้ setter
class BankAccountGood:
    def __init__(self, owner: str, balance: float = 0.0):
        self._owner  = owner
        self.balance = balance    # ← ผ่าน setter: validation ทำงาน

# ทดสอบ
bad  = BankAccountBad("Alice", -500.0)   # ไม่มี error — balance ติดลบได้
good = BankAccountGood("Alice", -500.0)  # ValueError ทันที
```

กฎง่ายๆ คือ ถ้ามี setter แล้วก็ควรใช้มันทุกที่ รวมถึงใน __init__ เพื่อให้ validation ทำงานสม่ำเสมอโดยไม่มีข้อยกเว้น

สรุป

- @property อย่างเดียวใช้เมื่อต้องการให้อ่านค่าได้แต่ไม่ให้แก้ไข และค่านั้นอาจมีการคำนวณเบื้องหลัง
- @property คู่กับ setter ใช้เมื่อต้องการให้แก้ไขได้แต่ต้องมีการตรวจสอบความถูกต้องก่อนเสมอ
- read-only property ใช้เมื่อข้อมูลนั้นถูกกำหนดครั้งเดียวแล้วไม่ควรเปลี่ยนแปลงอีก
- การเข้าใจความแตกต่างและเลือกใช้ให้ถูกสถานการณ์คือทักษะสำคัญของการออกแบบ class ที่ดี

ฝึกเขียนโปรแกรม



โจทย์

- เราจะสร้าง BankAccount ใหม่ตั้งแต่ต้นโดยคิดเรื่อง Encapsulation ตั้งแต่ขั้นตอนออกแบบ
- ก่อนเขียนโค้ดบรรทัดแรก เราต้องตอบคำถามสำคัญให้ได้ก่อนว่า "ข้อมูลแต่ละชิ้นควรถูกป้องกันในระดับใด และใครควรมีสิทธิ์แก้ไขได้บ้าง"
- ขั้นตอนที่ 1 — วิเคราะห์ข้อมูลก่อนเขียนโค้ด

ข้อมูล	คำอธิบาย	ควรเป็นแบบใด
account_number	หมายเลขบัญชี กำหนดตอนเปิดบัญชี ไม่ควรเปลี่ยนภายหลัง	read-only property
owner	ชื่อเจ้าของไม่ควรแก้ไขได้โดยตรงจากภายนอก	read-only property
balance	เปลี่ยนได้แต่ต้องไม่ติดลบ	property + setter พร้อม validation
transaction_count	ระบบนับเอง ไม่ควรให้ภายนอกตั้งค่า	read-only property

constructor

```
def __init__(self, account_number, owner, balance= 0.0):
```

- account_number คือ หมายเลขบัญชีในรูปแบบ string เก็บลงใน _account_number โดยตรง
  - ไม่ผ่าน setter เพราะไม่มี setter
- owner คือ ชื่อเจ้าของบัญชี เก็บลงใน _owner โดยตรงเช่นกัน
- balance คือ ยอดเงินเริ่มต้น มีค่า default เป็น 0.0 และต้องกำหนดผ่าน self.balance = balance
  - เพื่อให้ setter ตรวจสอบความถูกต้องตั้งแต่ตอนสร้าง object
- _transaction_count กำหนดเริ่มต้นเป็น 0 เสมอ ไม่รับจากภายนอก เพราะบัญชีใหม่ยังไม่มีธุรกรรมใดๆ

constructor : เฉลย

```
def __init__(self, account_number, owner, balance= 0.0):
    self.__account_number = account_number
    self.__owner = owner
    self.balance = balance
    self.__transaction_count = 0
```

Properties

- account_number – read-only คืนค่า _account_number
  - ไม่มี setter
- owner – read-only คืนค่า _owner
  - ไม่มี setter
- balance – มีทั้ง getter และ setter โดย
  - getter คืนค่า _balance
  - setter ตรวจสอบ 2 เงื่อนไขตามลำดับ คือ
    - value ต้องเป็น int หรือ float ก่อน หากไม่ใช่ให้ raise TypeError("ยอดเงินต้องเป็นตัวเลข")
    - จากนั้นตรวจว่า value ต้องไม่ติดลบ หากติดลบให้ raise ValueError("ยอดเงินต้องไม่ติดลบ")
    - เมื่อผ่านทั้งสองเงื่อนไขจึงบันทึกลง _balance
- transaction_count – read-only คืนค่า _transaction_count
  - ไม่มี setter

Properties : เฉลย

```python
#read only property
@property
def account_number(self):
    return self.__account_number

@property
def owner(self):
    return self.__owner

@property
def transaction_count(self):
    return self.__transaction_count
```

```python
#property + setter
@property
def balance(self):
    return self.__balance

@balance.setter
def balance(self, value):
    if not isinstance(value, (int, float)):
        raise TypeError("ยอดเงินต้องเป็นตัวเลข")

    if value < 0:
        raise ValueError("ยอดเงินต้องไม่ติดลบ")

    self.__balance = value
```

Method

- deposit(amount)
  - ตรวจสอบ amount ก่อนดำเนินการเสมอ
    - โดยตรวจว่า amount ต้องเป็น int หรือ float
      - หากไม่ใช่ให้ raise TypeError("จำนวนเงินต้องเป็นตัวเลข")
    - จากนั้นตรวจว่า amount ต้องมากกว่า 0
      - หากไม่ใช่ให้ raise ValueError("จำนวนเงินต้องมากกว่า 0")
    - เมื่อผ่านการตรวจสอบแล้วให้เพิ่มยอดเงินผ่าน self.balance += amount
    - และเพิ่ม _transaction_count ขึ้น 1 โดยตรง

Method : เฉลย

```python
def deposit(self, amount):
    if not isinstance(amount, (int, float)):
        raise TypeError("จำนวนเงินต้องเป็นตัวเลข")

    if amount <= 0:
        raise ValueError("จำนวนเงินต้องมากกว่า 0")

    self.balance += amount

    self.__transaction_count += 1
```

Method (2)

- withdraw(amount)
- ตรวจสอบ amount ก่อน
  - ตรวจ type ก่อน
  - จากนั้นตรวจว่า amount ต้องมากกว่า 0
  - และตรวจเงื่อนไขสุดท้ายว่า amount ต้องไม่เกิน self.balance ปัจจุบัน
    - หากเกินให้ raise ValueError("ยอดเงินไม่เพียงพอ")
  - เมื่อผ่านทั้งหมดให้ลดยอดเงินผ่าน self.balance -= amount และเพิ่ม _transaction_count ขึ้น 1

Method (2)

```python
def withdraw(self, amount):
    if not isinstance(amount, (int, float)):
        raise TypeError("จำนวนเงินต้องเป็นตัวเลข")

    if amount <= 0:
        raise ValueError("จำนวนเงินต้องมากกว่า 0")

    if amount > self.balance:
        raise ValueError("ยอดเงินไม่เพียงพอ")

    self.balance -= amount

    self.__transaction_count +=1
```

Method (3)

- display_info()
- แสดงข้อมูลบัญชีโดยเข้าถึงทุกค่าผ่าน property เท่านั้น
- ห้ามเข้าถึง _account_number, _owner, _balance, _transaction_count โดยตรง
- เพื่อให้โค้ดภายใน class เคารพ interface ที่ออกแบบไว้เช่นเดียวกับโค้ดภายนอก

Method (3)

```
def display_info(self):
    print(f"[{self.account_number}] {self.owner} | "
          f"ยอดเงิน: {self.balance:.2f} บาท "
          f"| ธุรกรรม: {self.transaction_count} ครั้ง")
```

การทดสอบการทำงานของ class

```python
acc = BankAccount("ACC-0001", "Alice", 1000)

# read-only properties
print(acc.account_number)     # ACC-0001
print(acc.owner)              # Alice
print(acc.balance)            # 1000
print(acc.transaction_count)  # 0

# deposit
acc.deposit(500)
print(acc.balance)            # 1500
print(acc.transaction_count)  # 1

# withdraw
acc.withdraw(200)
print(acc.balance)            # 1300
print(acc.transaction_count)  # 2
```

```python
# display
acc.display_info()
# → [ACC-0001] Alice | ยอดเงิน: 1300.00 บาท | ธุรกรรม: 2 ครั้ง
```

```python
# การป้องกันที่ต้องเกิดขึ้น
acc.balance = 999999
acc.owner = "Bob"
acc.account_number = "HACK"
acc.transaction_count = 0
acc.deposit(-100)
acc.withdraw(9999)
```

ข้อควรระวัง

- ใน _init_ บรรทัด self.balance = balance ควรเรียกผ่าน setter ไม่ใช่ self._balance = balance โดยตรง
  - เพื่อให้ validation ทำงานตั้งแต่ตอนสร้าง object
- transaction_count ต้องเพิ่มค่าจากภายใน method เท่านั้น โดยเข้าถึง self._transaction_count โดยตรงจาก method ภายใน class
  - ไม่ใช่ผ่าน setter เพราะเราไม่ต้องการให้โค้ดภายนอกมีสิทธิ์ตั้งค่านี้ได้เลย
- display_info() ควรเขียนว่า self.balance และ self.owner ไม่ใช่ self._balance และ self._owner
  - เพื่อแสดงให้เห็นว่าแม้แต่โค้ดภายใน class เองก็ควรเคารพ interface ที่ออกแบบไว้

Common Mistakes

ผิดพลาดที่ 1: สลับลำดับ property กับ setter

ผิด: setter ต้องมาหลัง property เสมอ

```
class BankAccount:
    @balance.setter    ← NameError: name 'balance' is not defined
    def balance(self, value):
        self.__balance = value

    @property
    def balance(self):
        return self.__balance
```

ถูก: property (getter) ต้องนิยามก่อนเสมอ

```
class BankAccount:
    @property
    def balance(self):
        return self.__balance

    @balance.setter    ← ใช้ชื่อ property ที่นิยามไว้แล้ว
    def balance(self, value):
        self.__balance = value
```

ผิดพลาดที่ 2: ใช้ @property.setter แทน @balance.setter

ผิด

```
@property.setter          ← AttributeError: 'property' object has no attribute
'setter'
def balance(self, value):
    self.__balance = value
```

ถูก : ใช้ชื่อ property นั้นๆ นำหน้า .setter

```
@balance.setter
def balance(self, value):
    self.__balance = value
```

ผิดพลาดที่ 3: ใน setter ยังเข้าถึง public attribute แทน private

ผิด : setter เขียนกลับไปที่ public attribute แทน

```
@balance.setter
def balance(self, value):
    if value < 0:
        raise ValueError("...")
    self.balance = value
```

← RecursionError: setter เรียกตัวเองวนไม่สิ้นสุด!

ถูก : setter เขียนกลับไปที่ private attribute เสมอ

```
@balance.setter
def balance(self, value):
    if value < 0:
        raise ValueError("...")
    self.__balance = value
```

← เขียนที่นี่เท่านั้น



ผิดพลาดที่ 4: พยายามตั้งค่า read-only property

```
account = BankAccount("Alice", 1000.0)
account.owner = "Bob"
```

AttributeError: can't set attribute 'owner'

ถ้า property ต้องการให้เขียนค่าได้ ต้องเพิ่ม @owner.setter เสมอ การไม่มี setter ไม่ใช่ bug แต่เป็นการออกแบบโดยเจตนา

ผิดพลาดที่ 5: สับสน raise กับ print ใน setter

ผิด — setter ใช้ print แล้ว return

```
@balance.setter
def balance(self, value):
    if value < 0:
        print("ค่าผิด")
        return
    self.__balance = value

# ทดสอบ
account.balance = -100
account.balance = 500
```

setter return โดยไม่ set ค่า → self.__balance ไม่ถูกอัปเดต

แต่ถ้า __balance ไม่เคยถูกสร้างในแรก → AttributeError เมื่อเรียก getter

Recap

- class ที่ดีไม่ได้หมายถึงเพียงการรวม attribute และ method ไว้ด้วยกัน แต่คือการควบคุมว่าข้อมูลภายในจะถูกเข้าถึงและแก้ไขได้อย่างไร ผ่านหลักการที่เรียกว่า Encapsulation
- จุดเริ่มต้นของหลักการนี้คือ access modifiers ซึ่งแบ่งระดับการเข้าถึงออกเป็น 3 ระดับ ได้แก่
  - public ที่เข้าถึงได้จากทุกที่
  - protected ที่ขึ้นต้นด้วย underscore 1 ตัว ซึ่งเป็นเพียงสัญญาณเตือนผู้อ่านโค้ดว่าไม่ควรแตะจากภายนอก
  - และ private ที่ขึ้นต้นด้วย underscore 2 ตัว ซึ่ง Python จะทำ name mangling ให้โดยอัตโนมัติ
- เมื่อ attribute เป็น private แล้ว @property คือ กลไกที่ช่วยให้โค้ดภายนอกยังสามารถอ่านค่าได้ด้วย syntax ที่เป็นธรรมชาติ โดยไม่ต้องเรียกผ่านวงเล็บ
- เมื่อต้องการให้แก้ไขค่าได้แต่ต้องผ่านการตรวจสอบก่อนเสมอ setter คือ จุดศูนย์กลางที่รวม validation logic ไว้ในที่เดียว ทำให้มั่นใจได้ว่าทุกการเปลี่ยนแปลงค่าต้องผ่านจุดนี้โดยไม่มีข้อยกเว้น
- สำหรับข้อมูลที่ควรกำหนดเพียงครั้งเดียวและไม่ควรเปลี่ยนแปลงอีก read-only property ที่ไม่มี setter คือเครื่องมือที่เหมาะสมที่สุด
- สิ่งที่สำคัญกว่าการจำ syntax คือการตัดสินใจเลือกระดับการเข้าถึงให้เหมาะกับบทบาทของข้อมูลแต่ละชิ้น ซึ่งเป็นทักษะที่สะท้อนความเข้าใจในการออกแบบ class อย่างแท้จริง

Bridge to Week 3

BankAccount ทำงานได้ดีสำหรับบัญชีทั่วไป แต่ในระบบธนาคารจริงมีบัญชีหลายประเภท เช่น บัญชีออมทรัพย์ที่มีดอกเบี้ย บัญชีกระแสรายวันที่มีวงเงินเบิกเกิน และบัญชีเงินฝากประจำที่มีเงื่อนไขการถอน

ถ้าจะรองรับทุกประเภทในคลาสเดียว จะเป็นแบบนี้...

```python
class BankAccount:
    def __init__(self, account_type: str, owner: str, balance: float):
        self.__account_type = account_type
        ...

    def withdraw(self, amount: float) -> None:
        if self.__account_type == "savings":
            # logic สำหรับบัญชีออมทรัพย์
        elif self.__account_type == "current":
            # logic สำหรับบัญชีกระแสรายวัน
        elif self.__account_type == "fixed":
            # logic สำหรับบัญชีเงินฝากประจำ
```

- ยิ่งเพิ่มประเภทบัญชีมากเท่าไหร่ก็ยิ่งซับซ้อนและดูแลยากขึ้น
- หากต้องการเพิ่มประเภทใหม่ก็ต้องแก้ไข method เดิมทุกครั้ง ซึ่งเป็นสัญญาณที่ชัดเจนว่าการออกแบบนี้ไม่ได้ scale ได้ดีพอ
- คำถามคือ มีวิธีออกแบบที่ทำให้แต่ละประเภทบัญชีมี logic เป็นของตัวเองได้ โดยไม่ต้องยัดทุกอย่างรวมไว้ใน class เดียว และยังคงใช้ประโยชน์จากโค้ดส่วนที่เหมือนกันร่วมกันได้อยู่หรือไม่
- สัปดาห์หน้าจะตอบคำถามนี้ด้วยหลักการ Inheritance ซึ่งเป็นกลไกที่ช่วยให้ class ใหม่สามารถ "ต่อยอด" จาก class ที่มีอยู่แล้วได้ โดยรับ attribute และ method ทั้งหมดมาโดยอัตโนมัติ และสามารถเพิ่มหรือปรับพฤติกรรมเฉพาะของตัวเองได้อย่างอิสระ



THANK YOU!